

Step 5 Results Note — End-to-End JSON Integration

The integration contract, the stitching pipeline, a bug found in it, and a full held-out evaluation revealing a real generalization gap

Problem ID	ML-T2-022
Candidate	Gyimah Ramsey Opoku
Student ID	9021623
Institution	KNUST, Kumasi, Ghana
Project Theme	Recovering Structure From Damaged or Poorly Scanned Documents
Stage	Stage 2 — Development, Step 5 of 5
Date	20 September 2026

1. Purpose

Steps 1 through 4 built and validated each pipeline stage separately: OCR under degradation, torn-region masking, masked-word reconstruction, and confidence-based uncertainty flagging. Step 5 combines all of them into a single per-page JSON output and checks that the combination actually works — not just on one hand-picked example, but across the full held-out test set, with a fixed, documented denominator.

Two things were treated as open questions rather than assumptions going in: whether the stitching code itself would introduce new bugs the individual pieces didn't have, and whether the confidence threshold locked in Step 4 (calibrated on pure text, no image, no OCR noise) would still behave the same way once real OCR noise entered the picture. Both questions turned out to matter.

2. The Integration Contract

A formal JSON Schema (`document_schema.json`, JSON Schema draft 2020-12) defines what every pipeline component must produce and, just as importantly, what it must not invent. Each page's output includes: a document/page identifier and source path; per-word OCR text, bounding box, and OCR confidence; damage status and fraction from the mask; reconstructed text and confidence where reconstruction was attempted; the uncertain flag and the threshold it was computed against; the final stitched text; per-word provenance (ocr or reconstructed); and evaluation metadata, present only when ground truth is available.

The schema enforces its own rules structurally, not just in prose: conditional (if/then) clauses require an intact word to carry provenance "ocr" with null reconstruction fields, and require a damaged word to carry provenance "reconstructed" with all reconstruction fields populated. This was tested directly by constructing a deliberately invalid document and confirming the schema rejects it, rather than trusting the description alone.

The schema's own description field states its scope honestly: `damage_status` and `damage_fraction` are computed by comparing a word's box against a mask this pipeline generated itself, because the pipeline knows where the damage is only because it created it for benchmarking. No component here detects damage location in a genuinely unknown scanned document with no reference mask; that remains unbuilt, and is named as future work rather than left implicit.

3. The Stitching Pipeline, and a Bug It Introduced

build_document_json.py calls the existing OCR, masking, and reconstruction functions in sequence and assembles their output into the schema's shape. Two simplifications were necessary and are documented in the code rather than hidden: the true number of RoBERTa subword tokens a damaged word needs is unknown without ground truth, so reconstruction tries 1, 2, and 3 mask tokens and keeps whichever has the highest confidence; and adjacent damaged words reconstruct independently, with a placeholder standing in for a still-damaged neighbor rather than that neighbor's own (not-yet-computed) reconstruction.

The first simplification produced a real bug on the very first test run. A partially torn word ("solutions;") was "reconstructed" as "choices, and" — two words and a comma, not one word. The 3-mask-token search had found a token sequence with genuinely high model confidence, but nothing constrained the result to actually be a single word: RoBERTa's subword tokens don't respect word boundaries, so three tokens can legitimately spell out a short phrase spanning what should have been one word's slot.

The fix: reject any candidate reconstruction whose decoded text contains a space, before comparing confidences across the 1/2/3-token hypotheses. Re-run on the same input, the same word now reconstructs to "choices;" — one word, still wrong, but structurally valid, with its confidence correctly dropping from 0.193 to 0.107 once the invalid higher-confidence candidate was excluded from consideration.

A separate, cosmetic anomaly was investigated rather than assumed away: a missing space appeared in a printed evaluation string, but at a different word pair on each of two otherwise-identical runs. A moving gap under identical, deterministic code rules out a real bug in the join logic (which is a plain, deterministic Python operation) and points to a terminal line-wrap artifact from copying long JSON lines out of PowerShell. It was confirmed this way rather than patched blindly, since patching a non-bug would have hidden the real explanation.

4. Denominator Reconciliation

Before running any batch evaluation, the held-out test set size was re-verified directly from the file on disk rather than trusted from memory. A short script hashed sentence_corpus.jsonl (SHA-256: 937dd356...) and counted its split == "test" records at run time: 739. This matches every figure used since Step 4 and contradicts an earlier, unverified recollection of 757 — most plausibly a stale count from before build_corpus.py's chapter-heading filter was added, though that specific origin was not confirmed and is not asserted as fact.

This run's own evaluation uses a third, distinct N, and the distinction is deliberate, not an inconsistency to reconcile away. Step 4's 1,478-example benchmark was pure text: two words masked per sentence, no image, no OCR, no tear. This Step 5 evaluation instead renders each of the 739 held-out sentences to an image and deliberately damages exactly one randomly chosen word per sentence (using the same word-targeting approach validated in Step 2/3), so that every document contributes exactly one reconstruction attempt. That makes N = 739 damaged-word instances here, not 1,478 — a smaller, differently-constructed set measuring a different thing: the integrated pipeline under real image and OCR conditions, rather than the reconstruction model in isolation under clean text.

5. Full Held-Out Evaluation

All 739 held-out sentences were processed: rendered, targeted-torn, OCR'd, damage-classified, reconstructed, confidence-scored against the Step 4 threshold (0.5442, applied unchanged, not re-tuned here), assembled into a schema-validated JSON per document, and written to disk. All 739 succeeded; zero errors; every document validated against document_schema.json.

Metric	Value
OCR-only CER (before reconstruction)	0.0492

Metric	Value
OCR-only WER	0.0560
Final stitched CER (after reconstruction)	0.0300
Final stitched WER	0.0383

Table 1. Corpus-level (pooled, not per-document-averaged) CER/WER, n = 739 documents.

Metric	Value
Reconstruction top-1 accuracy, overall (n = 781 words)	40.32%
Reconstruction top-1 accuracy, content words (n = 521)	28.02%

Table 2. Whole-word reconstruction accuracy across all damaged words in the held-out set.

Metric	Step 4 (text-only, n = 1,478)	This run (real image, n = 781 reconstructed words)
Flag rate	50.20%	46.55%
Error recall	77.21%	65.76%
Precision	78.98%	84.30%
Unflagged error rate	~23.5%	38.23%

Table 3. The Step 4 locked threshold applied, unchanged, to this run's real-image distribution, compared to its original text-only benchmark.

Provenance	Word count
OCR (intact)	14,899
Reconstructed (damaged)	781
Total	15,680

Table 4. Word counts by provenance across all 739 documents.

6. Interpretation

Reconstruction still helps at the corpus level: stitching in reconstructed words for damaged positions cut CER by nearly 40% (0.0492 to 0.0300) and WER by roughly a third (0.0560 to 0.0383) relative to leaving OCR's raw, damaged-word output in place. That benefit is real and worth keeping.

The more important finding is that the confidence threshold locked in Step 4 does not transfer cleanly to this distribution. Recall dropped from 77.21% to 65.76% — the same threshold now catches noticeably fewer of the actual errors. Precision rose, from 78.98% to 84.30%, so what it does flag is more reliably wrong. But the number that carries the real practical claim — how much an archivist can trust the unflagged output — got substantially worse: unflagged error rate nearly doubled, from roughly 23.5% in Step 4's clean-text benchmark to 38.23% here. Real OCR noise in the surrounding context, which Step 4's pure text-masking setup never introduced, measurably degrades the confidence signal's practical value. This is a genuine generalization gap, not an implementation defect, and it should be reported as such rather than smoothed over.

Reconstruction accuracy also shifted in a way that is informative rather than merely disappointing: overall accuracy fell more (48.65% to 40.32%) than content-word accuracy did (27.03% to 28.02%, effectively unchanged). This is consistent with function words being more sensitive to OCR-corrupted surrounding context than content words are — content-word reconstruction was already difficult under clean-text conditions, so it had less room to get worse.

7. Next Steps

- Recalibrate the confidence threshold directly on this real-image distribution (repeat Step 4's validation-only calibration procedure, but on rendered-and-torn images rather than pure text), rather than assuming a text-calibrated threshold generalizes.
- Consider combining OCR confidence with reconstruction confidence as a joint uncertainty signal, since OCR noise in the surrounding context is exactly what degraded the current threshold's performance.
- Address the real-deployment damage-detection gap named in Section 2: nothing in this pipeline yet locates damage in a scan with no known mask, which remains the boundary between this benchmark and an actual field-deployable system.
- The per-document JSON outputs, manifest, and summary report from this run are retained (held_out_documents/, held_out_manifest.json, held_out_summary_report.json) so this evaluation can be reproduced or extended without re-deriving the held-out set from scratch.